

UnifAI Enterprise Deep-Dive Architecture & Feature Master Guide

விரிவான கணினி கட்டமைப்பு, ஹெடர்ஸ் மற்றும் 38 அம்சங்களின் மாழாமயைான கையேடு (Tamil & English Technical Manual)

1. Executive Summary & Core Platform Overview

UnifAI is a high-performance Unified AI Gateway, Router, Governance, and Observability Control Plane. It centralizes AI consumption across OpenAI, Anthropic Claude, Google Gemini, AWS Bedrock, Mistral, and local models. It delivers up to 80% cost reduction via Complexity Routing and Semantic Caching, 99.99% uptime via Circuit Breakers, comprehensive governance with Virtual Keys ('sk-uf-*'), and pre/post-LLM content guardrails.

2. End-to-End 7-Stage Request Lifecycle

- Transport & Governance: FastHTTP validates Virtual Key ('sk-uf-*'), enforces rate limits and budget quotas.
- Pre-LLM Guardrails: Evaluates Google CEL rules and Presidio DLP to redact PII and block prompt injections.
- Semantic Caching: Vector similarity search in Redis/Qdrant. Cache hit returns immediately (<20ms, \$0 cost).
- Intelligent Routing: Complexity Router classifies prompt (Simple/Reasoning); Circuit Breakers check provider health.
- Upstream Execution: Dispatches prompt to target LLM or executes MCP tools with OAuth user credentials.
- Post-LLM Guardrails: Scans generated output for toxic content or credential leaks before returning.
- Observability & Telemetry: Appends record to LLM Logs, streams to Datadog/Kafka, and returns response with 'x-unifai-*' headers.

3. Exhaustive HTTP Headers Reference Guide

Authentication & Identity Headers (அடையாளம் & பாதுகாப்பு)

Header Name	Sample Value	Type	Description (English & Tamil)
x-uf-vk	sk-uf-prod_corp_a91b2c3d4e	String (Virtual Key)	UnifAI Virtual Key representing synthetic API credentials. Binds the caller to an authorized Team, User, Budget Quota, Access Profile, and MCP Tool Groups. [தமிழ்] Virtual Key மூலமாக client அங்கீகரிக்கப்படுகிறது. Upstream provider keys மறைக்கப்பட்டு, பட்ஜெட் மற்றும் அனுமதிகள் இதனுடன் இணைக்கப்படும்.
Authorization	Bearer sk-uf-prod_corp_a91b2c3d4e	Standard Header	Standard HTTP Authorization header alternative for OpenAI SDK compatibility. Automatically parsed as a Virtual Key if prefixed with 'sk-uf-'. [தமிழ்] OpenAI SDK பயன்படுத்தும்போது 'x-uf-vk'-க்கு மாற்றாக standard Bearer token வடிவத்தில் Virtual Key-ஐ அனுப்பலாம்.
x-uf-api-key	key_adm_89f0a21bc9e4	String (Admin Key)	Platform Control Plane Administrative API Key. Authorizes access to administrative CRUD endpoints (creating keys, editing routing rules, cluster sync). [தமிழ்] UnifAI நிர்வாக API-களை அணுக பயன்படும் Platform Admin Key. பாதிய ரூல்ஸ், மாதல்கள் மற்றும் கீகளை உரவாக்க உதவும்.
x-uf-customer-id	cust_enterprise_7710	String (UUID/Slug)	Tenant / Customer scoping key for B2B multi-tenant SaaS applications. Routes usage, token consumption, and dollar spend to a specific end-customer account under a shared team virtual key. [தமிழ்] B2B SaaS செயலிகளில், எந்த இறுதி வாடிக்கையாளர் (End Customer) இந்த request-ஐ அனுப்புகிறார் என பிரித்து Cost & Analytics track செய்ய உதவும்.
x-uf-customer-name	Acme Corporation Global	String (Human Name)	Display name for the customer tenant, populated in audit records, billing statements, and telemetry dashboards alongside x-uf-customer-id. [தமிழ்] வாடிக்கையாளரின் பெயர். Audit logs மற்றும் பில்லிங் அறிக்கைகளில் எளிதாக அடையாளம் காண பயன்படுகிறது.
x-uf-direct-key	true / false	Boolean Flag	Direct Key Pass-Through. When set to 'true', UnifAI bypasses its internal key pool and uses the raw provider key supplied in standard headers (e.g. x-api-key or x-goog-api-key). [தமிழ்] UnifAI-ல் சமீகிக்கப்பட்ட key-ஐ பயன்படுத்தாமல், Client தன்னிடம் உள்ள raw OpenAI/Anthropic key-ஐ நேரடியாக பாஸ் செய்ய உதவும்.
X-UnifAI-Temp-Token	jwt_or_uuid_token	Short-lived Session Token	Ephemeral temporary session token used during interactive OAuth consent flows, MCP tool authorization callbacks, and restricted UI views. [தமிழ்] MCP டூல்ஸ் மற்றும் OAuth login callbacks-ன் போது தற்காலிக அங்கீகாரத்திற்காக பயன்படுத்தப்படும் short-lived டோக்கன்.

Intelligent Routing & Model Steering Headers (ரூட்டிங் & மாடல் தரேவ்)

Header Name	Sample Value	Type	Description (English & Tamil)
x-uf-provider	openai anthropic bedrock vertex groq ollama	Provider Enum	Explicit Provider Pinning. Overrides automated routing rules and forces UnifAI to dispatch the request directly to the specified upstream model provider. [தமிழ்] தானியங்கி ரூட்டிங் விதிகளை தவிர்த்து, காறிப்பிட்ட ஒரு AI Provider-க்கு (Ex: Anthropic அல்லது Bedrock) நேரடியாக கோரிக்கையை அனுப்ப உதவும்.
x-uf-model	claude-3-7-sonnet gpt-4o deepseek-r1	Model ID	Explicit Model Selection. Overrides the body 'model' parameter or acts as the primary model target when using native SDK conversions. [தமிழ்] கோரிக்கை இயக்கப்பட வணேடிய மாடலின் பெயரை நேரடியாக நிர்ணயிக்க பயன்படுகிறது.
x-uf-api-key-id	key_azure_eastus_02	Credential Pool ID	Target Credential Pinning. When a single provider has multiple keys (e.g., 5 Azure OpenAI endpoints in different regions), pins execution to a specific key ID. [தமிழ்] ஒரே provider-ல் பல API keys (multi-region accounts) இரூக்கம்போது, காறிப்பிட்ட ஒரு key-ஐ மட்டும் தரேவ் செய்ய உதவும்.
x-uf-circuit-breaker-bypass	true	Diagnostic Flag	Administrative Circuit Breaker Bypass. Forces the gateway to attempt dispatch to an upstream provider even if its circuit breaker is currently in the TRIPPED (OPEN) state. [தமிழ்] Circuit Breaker open-ஆக (tripped) இரூந்தாலும், சோதனைகளுக்காக அந்த provider-க்கு கட்டாயமாக request அனுப்ப உதவும்.

Semantic Caching & Performance Headers (கச்சேசிங் & லடேன்சி காற்றைப்பு)

Header Name	Sample Value	Type	Description (English & Tamil)
x-uf-cache-key	tenant_alpha:customer_support:v2	Namespace String	Cache Partition Namespace. Segregates cached response vectors into isolated buckets to guarantee complete data isolation between tenants, departments, or application versions. [தமிழ்] கச்சே தரவை தனித்தனி பாகங்களாக (partitions) பிரிக்க உதவுகிறது. வெவ்வேறான வாபிக்கையாளர்களின் டட்டோ ஒன்றோடொன்று கலக்காமல் பாதுகாக்கும்.
x-uf-cache-ttl	3600 (seconds) 86400	Integer (Seconds)	Per-Request Cache Time-To-Live. Overrides the global cache TTL to specify exactly how many seconds this response should remain valid in the cache. [தமிழ்] இந்த காறிப்பிட்ட பதில் எத்தனை வினாடிகள் கச்சேசில் சமேகிக்கப்பட்டிருக்க வணேடும் என்பதை நிர்ணயிக்கிறது.
x-uf-cache-threshold	0.92 (range: 0.0 to 1.0)	Float Score	Cosine Similarity Threshold Override. Sets the minimum vector similarity required to consider a stored prompt a 'Cache Hit'. Higher = more accurate; Lower = more hits. [தமிழ்] Semantic cache-ல் இரண்டு களேவிகள் எத்தனை சதவீதம் ஒத்துப்போக வணேடும் (Cosine similarity) என்பதை மாற்ற உதவும்.
x-uf-cache-type	direct semantic	Cache Mode Enum	'direct' enforces exact SHA-256 prompt string matching. 'semantic' generates an embedding vector and performs approximate nearest neighbor (ANN) search. [தமிழ்] 'direct' என்பது 100% வார்த்தை பொருத்தம்; 'semantic' என்பது அர்த்த ரீதியான பொருத்தம் (Embedding Vector similarity).
x-uf-cache-no-store	true / false	Boolean Flag	No-Store Directive. When set to 'true', UnifAI will serve a cached response if a hit exists, but will NOT store the new LLM completion in the cache upon a miss. [தமிழ்] ஏற்கனவே கச்சேசில் இரூந்தால் பதிலை எடூ, ஆனால் பாதிய பதிலை கச்சேசில் சமேகிக்காதே என்ற உத்தரவு.

MCP (Model Context Protocol) & Tool Headers (டூல்ஸ் பயன்பாடூ)

Header Name	Sample Value	Type	Description (English & Tamil)
x-uf-mcp-include-clients	* github,postgres,slack	Comma-separated List	MCP Server Whitelist. Injects only the specified MCP servers into the LLM request. Setting '*' exposes all servers permitted by the caller's Access Profile. [தமிழ்] எந்தெந்த MCP servers-ன் டூல்ஸ்களை மாடலுக்கு வழங்க வணேடும் என கட்டப்படுத்தும் whitelist (Ex: github, postgres).
x-uf-mcp-include-tools	query_database,create_pull_request	Comma-separated List	MCP Tool Filter. Restricts the exposed tools to a specific subset of function names within the permitted MCP servers. [தமிழ்] சர்வரில் உள்ள டூல்ஸ்களில் காறிப்பிட்ட சில functions-ஐ மட்டும் மாடலுக்கு வெளிப்படுத்த உதவும்.
x-uf-mcp-session-id	sess_oauth_usr_8821a	Session Identifier	Per-User Tool Session Key. Binds the tool execution to a human user's personal OAuth session (e.g. user's personal GitHub token rather than a system token). [தமிழ்] பயனரின் தனிப்பட்ட OAuth credentials மூலம் MCP tools-ஐ இயக்க பயன்படும் Session ID.
x-uf-eh-*	x-uf-eh-workspace-id: ws_991	Prefixed Passthrough	Extra Tool Headers. Any header matching the 'x-uf-eh-*' prefix is forwarded directly to upstream MCP servers if included in their allowed_extra_headers configuration. [தமிழ்] MCP சர்வராக்கு தவேப்படும் தனிப்பயன் headers-ஐ பாதுகாப்பாக forward செய்ய உதவும் prefix.

Observability, Tracing & Debugging Headers (கண்காணிப்பு & பிழைத்திரத்தம்)

Header Name	Sample Value	Type	Description (English & Tamil)
x-uf-session-id	sess_chat_conv_12345	String (Session ID)	Conversation Session Grouping. Groups multiple sequential LLM calls into a single unified trace in OpenTelemetry and LLM Logs. [தமிழ்] ஒரு பயனரின் தொடர்ச்சியான சாட் உரையாடல்களை ஒரே Trace-ஆக இணைக்க பயன்படும் Session ID.
x-uf-dim-	x-uf-dim-environment: production	Arbitrary Key-Value	Custom Log Dimension. Injects custom dimensional metadata into internal log records and exported connector events (e.g., department, tier, feature). [தமிழ்] Logs-ல் custom filters சேர்க்க பயன்படும் metadata (Ex: x-uf-dim-app: mobile-app).
x-uf-lh-	x-uf-lh-correlation-id: req_091	Header Capture	Captured Request Header. Explicitly instructs UnifAI's logging engine to record the value of this header into the LLM log entry. [தமிழ்] காறிப்பட்ட ஒரு request header-ஐ logs-ல் பதிவு செய்ய கட்டளையிடும் header.
x-uf-disable-content-logging	true / false	Privacy Flag	Content Privacy Mode. Completely drops prompt and completion text from log storage, recording only token counts, duration, latency, cost, and metadata. [தமிழ்] ரகசியத்தன்மையை பாதுகாக்க, prompt மற்றும் பதிலை logs-ல் சேமிக்காமல், வறொம் செலவா மற்றும் வகேத்தை மட்டும் பதிவு செய்ய உதவும்.
x-uf-store-raw-request-response	true / false	Debug Capture	Raw Wire Byte Storage. Instructs the gateway to persist the exact raw JSON byte stream exchanged with the upstream provider for low-level debugging. [தமிழ்] Upstream provider-க்கு அனுப்பிய அசல் raw bytes-ஐ logs-ல் சேமிக்க உதவும் debug toggle.
x-uf-send-back-raw-request	true / false	Echo Header	Debug Echo. Injects the converted raw request payload sent to the upstream provider into a debugging HTTP response header. [தமிழ்] Gateway மாற்றி அனுப்பிய raw payload-ஐ response header-ல் திரும்பப் பறெ உதவும்.
x-uf-send-back-raw-response	true / false	Echo Header	Debug Echo. Injects the unparsed upstream response payload into a debugging HTTP response header. [தமிழ்] Upstream provider தந்த அசல் பதிலை response header-ல் திரும்பப் பறெ உதவும்.
traceparent	00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01	W3C Trace Context	Distributed Tracing Header. Propagates distributed OpenTelemetry trace context from calling microservices through UnifAI to downstream connectors. [தமிழ்] Microservices வழியே OpenTelemetry Distributed Tracing-ஐ தொடர்ந்து கண்காணிக்க உதவும் W3C standard header.
x-uf-log-repo-id	repo_maxim_eval_01	External Repo ID	Evaluation Routing. Directs the request's telemetry log directly to a designated evaluation repository in platforms like Maxim AI. [தமிழ்] Maxim AI போன்ற வளனி evaluation தளங்களில் காறிப்பட்ட repository-க்கு logs-ஐ அனுப்ப உதவும்.

Gateway Response Headers (UnifAI திராப்பி அனுப்பும் Headers)

Header Name	Sample Value	Type	Description (English & Tamil)
x-unifai-provider	anthropic openai bedrock	Provider String	Indicates which model provider actually executed and served the request. [தமிழ்] எந்த AI Provider கரோரிக்கயை நிறவைறேறியது என்பதை வாடிக்கையாளர்க்கு தறிவிக்கும்.
x-unifai-original-model	gpt-4o	Model Identifier	The model originally requested by the client application prior to routing. [தமிழ்] வாடிக்கையாளர் கேட்ட அசல் மாடலின் பெயர்.
x-unifai-resolved-model	claude-3-5-sonnet	Model Identifier	The actual model architecture that handled the prompt following routing rule or complexity tier resolution. [தமிழ்] Routing விதிகள் மற்றும் complexity ஆய்வாக்கப் பிறகு உண்மையில் இயங்கிய மாடல்.
x-unifai-fallback-index	0 (primary) 1, 2... (fallback)	Integer Index	Indicates whether a fallback route was taken. '0' denotes primary provider success; non-zero indicates which fallback triggered. [தமிழ்] Primary provider தரோல்வியடறந்து Fallback provider இயங்கியதா என்பதை காட்டும் (0 = No fallback).
x-unifai-request-type	chat completion embedding realtime	Type String	Identifies the execution modality: chat completions, text completions, embeddings, or realtime voice. [தமிழ்] கரோரிக்கயின் வகை: chat, embeddings, realtime voice, etc.

4. In-Depth Technical Dissection of All 38 Features (7 Pillars)

Pillar 1: Observability (மாழாமயைான கண்காணிப்பு & செயல்திறன்)

The Observability pillar provides sub-millisecond telemetry, cost accounting, request logging, and external data connectors across all AI workloads.

Dashboard (/workspace/dashboard)

Overview: Real-time executive analytics console displaying financial, throughput, and performance metrics across models and virtual keys.

Internal Architecture: The Dashboard is powered by a high-frequency WebSocket connection (useWebSocket) paired with TanStack Query. In the backend, metrics are aggregated in-memory using atomic counters and persisted to time-series Postgres tables. It calculates p50, p90, p95, and p99 latency percentiles, tracks dollar spend using live Pricing Overrides, and measures cache hit ratios across all active tenant nodes.

- Time Range Filter: 1h, 24h, 7d, 30d, and custom calendar windowing.
- Spend Gauges: Total aggregate cost (\$), daily burn rate, and projected monthly invoice.
- Latency Percentiles: p50, p95, p99 curves showing TTFT (Time To First Token) and total latency.
- Breakdown Views: Bar and donut charts slicing traffic by Provider, Model, and Virtual Key.
- Cache Efficiency Meter: Real-time calculation of cost saved via Semantic Cache hits.

Interconnections: Receives aggregated telemetry from LLM Logs, Pricing Overrides, and Semantic Caching. Influences Budgets & Limits alert triggers.

Production Use Case: Engineering leaders and FinOps teams monitor daily token burn, identify slow endpoints (p99 > 3000ms), and detect abnormal traffic surges before budget overruns occur.

LLM Logs (/workspace/logs)

Overview: Full-fidelity request and response transaction ledger capturing complete prompts, completions, tokens, latency, and guardrail verdicts.

Internal Architecture: Requests passing through FastHTTP are buffered asynchronously into an in-memory queue (logstore) to guarantee zero impact on proxy latency. A dedicated background worker pool writes records in micro-batches to PostgreSQL with pgx v5. Each log entry captures request_id, session_id, virtual_key_id, provider, model requested vs resolved, prompt_tokens, completion_tokens, exact dollar cost, latency_ms, ttft_ms, HTTP status, and full JSON payloads.

- Full-Text & Regex Search: Search across user prompts and model completions.
- Multidimensional Filtering: Filter by Virtual Key, Customer ID, Provider, Model, Status Code, and Date Range.
- Content Privacy Control: Supports x-uf-disable-content-logging to omit sensitive text while preserving metrics.
- Raw Byte Inspector: Inspects exact wire JSON exchanged with upstream providers.
- Guardrail Annotation: Visual flags indicating whether input or output triggered PII redaction or safety rules.

Interconnections: Fed by every LLM invocation; powers the Dashboard metrics; streams out to Connectors (Datadog, Kafka); validates Guardrails accuracy.

Production Use Case: Developers troubleshoot failed user requests, investigate hallucinated model answers, verify TTFT performance, and export compliance audit logs.

MCP Logs (/workspace/mcp-logs)

Overview: Granular execution audit log tracking Model Context Protocol tool invocations, arguments, return payloads, and runtime errors.

Internal Architecture: Located in 'core/mcp/exec.go', the MCP interceptor instruments every tool execution. It records the calling agent's identity, the target MCP server, function name, parsed input JSON arguments, execution duration in milliseconds, and the exact tool output or error stack trace. Supports both stdio and SSE transport protocols.

- Tool Call Inspector: View structured input parameters and returned output schemas.
- Execution Time Tracking: Measure tool execution latency to identify slow database queries or API scrapers.
- Error Diagnostics: Full exception stack traces for crashed tool processes or timeout breaches.
- Session Binding: Correlates tool calls with specific User Auth Sessions and Virtual Keys.

Interconnections: Tied directly to the MCP Gateway, Tool Groups, and Auth Sessions. Streams tool performance into Dashboard.

Production Use Case: AI Agent engineers diagnose why an autonomous agent entered an infinite tool loop, verify SQL queries executed by the Database MCP tool, and track 3rd-party API failures.

Browser AI (/workspace/browser-ai)

Overview: Enterprise Browser Extension and local DLP proxy that intercepts employee AI queries on public web portals (ChatGPT, Claude) to enforce corporate data safety.

Internal Architecture: Consists of two synchronized layers: (1) A local proxy daemon (browser_ai_proxy.py) with browser extensions deployed via MDM, and (2) a centralized control console in the UnifAI UI. It intercepts HTTPS requests to web AI portals, runs Presidio DLP and local Ollama classification models, and blocks the transmission of proprietary source code, credentials, or customer PII.

- Target Websites Registry: Manages parent and sub-domain policies (e.g. chatgpt.com, claude.ai, perplexity.ai).
- DLP Rule Engine: Regex-based and LLM-assisted policy enforcement on browser paste events.
- Attachment Text Extractor: Inspects and extracts text from uploaded PDF, Word, and text attachments.
- Tamper-Proof Agent Management: Heartbeat tracking, bulk agent deletion, and uninstall key protection.
- Violation Logs: Dedicated log tracking employee DLP violations with source IP, user email, and matched policy.

Interconnections: Shares Guardrail Providers (Presidio, Regex) and routes violation telemetry into Audit Logs and Connectors.

Production Use Case: Prevents corporate data leakage by stopping engineers from pasting confidential customer databases or private cryptographic keys into public web ChatGPT.

Connectors (/workspace/observability)

Overview: Real-time streaming export pipelines forwarding UnifAI metrics, logs, and trace spans to enterprise observability platforms.

Internal Architecture: Implemented in 'framework/connectors/runtime.go'. It operates as an asynchronous worker ring buffer that decouples export operations from the gateway request path. It converts internal telemetry events into platform-native formats and delivers them with automatic retries, exponential backoff, and circuit-breaking.

- Datadog Connector: Exports APM trace spans and metrics (unifai.requests.count, unifai.tokens.total, unifai.latency).
- New Relic Connector: Streams custom events and transaction metrics for enterprise APM.
- Apache Kafka Connector: Publishes structured JSON event streams to dedicated enterprise Kafka topics.
- Google BigQuery Connector: Direct streaming inserts into data warehouse tables for custom SQL billing analytics.
- Google Cloud PubSub Connector: Pushes event notifications to downstream serverless functions.
- OpenTelemetry (OTel) Exporter: OTLP gRPC/HTTP exporter compatible with Prometheus, Jaeger, and Grafana.

Interconnections: Reads continuously from LLM Logs, MCP Logs, and Audit Logs; forwards data to external corporate SIEMs.

Production Use Case: Enterprises stream all AI consumption logs to their centralized Datadog dashboard and BigQuery data lake for corporate billing audits and security threat analysis.

Logs Settings (/workspace/config/logging)

Overview: Centralized policy management console for log retention, sampling ratios, data privacy redaction, and raw byte persistence.

Internal Architecture: Located in 'ui/app/workspace/config/views/loggingView.tsx'. Controls internal storage garbage collection routines, sampling filters in the FastHTTP middleware, and sensitive payload redaction algorithms.

- Retention Policy: Configurable auto-purge schedules (e.g. 7 days, 30 days, 90 days, 365 days).
- Traffic Sampling: Configures percentage-based sampling (1% to 100%) to reduce storage costs in massive volume environments.
- Content Redaction Toggle: Global enforcement of zero-prompt storage for extreme regulatory environments.
- Raw Wire Byte Storage: Toggle for storing full wire-level JSON payloads for diagnostic deep-dives.
- Storage Offloading: Integrates with AWS S3 and Google Cloud Storage for cold log archival.

Interconnections: Directly governs the storage behavior of LLM Logs and MCP Logs.

Production Use Case: Healthcare and financial institutions enforce 30-day auto-deletion policies and disable prompt storage to satisfy HIPAA and PCI-DSS compliance.

Pillar 2: Models & Intelligent Routing (மாதல் மலோண்மதை & ருட்டிங்)

The Models pillar orchestrates model cataloging, dynamic multi-provider traffic routing, financial budget quotas, and automated failover.

Model Catalog (/workspace/model-catalog)

Overview: Unified inventory of all connected models across cloud and self-hosted providers, displaying context limits, capabilities, and pricing.

Internal Architecture: Maintained in 'framework/modelcatalog'. Aggregates model metadata from all configured providers. Maintains token window boundaries, max generation tokens, supported modalities (text, vision, audio), feature capabilities (function calling, JSON mode, reasoning effort), and real-time provider reachability.

- Model Directory: Comprehensive searchable catalog of models from OpenAI, Anthropic, Bedrock, Vertex, Ollama, etc.
- Capability Matrix: Flags for Tool Calling, Vision, Structured Outputs, and Streaming support.
- Token Limits: Context window size (e.g. 128k, 200k, 1M) and maximum output token ceilings.
- Model Aliasing: Maps logical names (e.g. 'production-fast') to specific underlying architectures.

Interconnections: Supplies target definitions to Routing Rules, Complexity Router, and Access Profiles. Displays pricing in Dashboard.

Production Use Case: Developers query the catalog to discover which models support vision inputs and tool-calling capabilities across enterprise accounts.

Model Providers (/workspace/providers)

Overview: Credential and endpoint management console for upstream AI vendors, supporting multi-key rotation and custom private endpoints.

Internal Architecture: Located in 'core/providers'. Contains dedicated adapters for 25+ AI providers (OpenAI, Azure, Anthropic, Bedrock, Vertex, Groq, Cerebras, Cohere, DeepSeek, Mistral, Ollama, vLLM, Replicate). Implements connection pooling, custom base URLs, and multi-key rotation pools with round-robin or priority weighting.

- Provider Adapters: Specialized wire protocol implementations for each major AI provider.
- Multi-Key Rotation: Pool multiple API keys to distribute rate limits and avoid provider throttling.
- Private Endpoints: Configure custom base URLs for internal vLLM, Ollama, or VPC-peered deployments.
- Connection Tuning: Per-provider HTTP keep-alive, dial timeouts, and connection pool sizes.

Interconnections: Powers the Model Catalog; feeds health data to Circuit Breaker; executes requests dispatched by Routing Rules.

Production Use Case: Platform teams rotate OpenAI API keys without downtime and configure dedicated AWS Bedrock IAM role credentials.

Budgets & Limits (/workspace/model-limits)

Overview: Multi-tier rate limiting and financial budget enforcement engine preventing runaway spending and provider throttling.

Internal Architecture: Implemented in 'framework/configstore' and 'plugins/governance/tracker.go'. Maintains atomic sliding-window rate limit counters (RPM, TPM, RPD) in Redis / local memory. Continuously evaluates current token burn against daily and monthly dollar budgets (\$ USD). Triggers immediate rejection with HTTP 429 when limits are breached.

- Throughput Limits: Requests Per Minute (RPM), Tokens Per Minute (TPM), Requests Per Day (RPD).
- Dollar Spending Caps: Hard and soft monthly/daily financial budget limits in USD.
- Multilevel Scoping: Enforce limits globally, per Virtual Key, per Team, per User, or per Model.
- Breach Actions: Reject with 429 Too Many Requests, downgrade to a cheaper model, or trigger webhook alerts.

Interconnections: Evaluated during Stage 1 of the request lifecycle; updates budget balances from LLM Logs token consumption.

Production Use Case: Ensures an experimental hackathon project cannot exceed \$500/month or flood the company's shared OpenAI tier with 10,000 RPM.

Routing Rules (/workspace/routing-rules)

Overview: Condition-based dynamic routing engine powered by Google CEL expressions for intelligent traffic routing and A/B testing.

Internal Architecture: Located in 'plugins/governance/routing.go'. Pre-compiles Google Common Expression Language (CEL) rules into memory. Upon request arrival, evaluates rules against request context (model name, prompt content, user department, headers). Selects target provider, performs traffic splitting, or triggers fallback chains.

- CEL Expression Builder: Define rules like ``request.model == 'fast' && prompt.length < 500``.
- Traffic Splitting: Distribute traffic (e.g. 80% to OpenAI, 20% to Anthropic) for zero-risk model evaluation.
- Header-Based Routing: Route requests based on custom headers like ``x-uf-dim-env: staging``.
- Fallback Target Sequences: Ordered array of backup models if primary destination is unavailable.

Interconnections: Works in tandem with Complexity Router; hands off resolved target to Circuit Breaker.

Production Use Case: Automatically routes customer queries mentioning 'code' to DeepSeek Coder, while routing French queries to Mistral Large.

Complexity Router (/workspace/complexity-router)

Overview: Intelligent prompt complexity classifier that buckets queries into 4 tiers and routes them to cost-appropriate models, cutting costs by 60-80%.

Internal Architecture: Located in 'plugins/governance/complexity'. Analyzes incoming user messages across 4 distinct tiers: SIMPLE, MEDIUM, COMPLEX, and REASONING. Uses keyword dictionary matching, token length boundaries, and lightweight heuristics to identify task difficulty and route to corresponding model architectures.

- Tier Palette: Simple (P1), Medium (P2), Complex (P3), Reasoning (P4).
- Keyword Lists: Reasoning keywords ('prove', 'step by step', 'derivation'), coding terms, and simple greetings.
- Boundary Controls: Token length thresholds separating simple questions from heavy multi-turn reasoning.
- Tier Target Mapping: Simple -> gpt-4o-mini; Medium -> claude-3-5-haiku; Complex -> gpt-4o; Reasoning -> o1.

Interconnections: Feeds classified tier context directly into Routing Rules; updates cost metrics in Dashboard.

Production Use Case: Slashes enterprise AI bills by routing routine "What are your store hours?" questions to a \$0.15/1M token model instead of a \$15/1M token reasoning model.

Circuit Breaker (/workspace/circuit-breaker)

Overview: Automated failure detection and failover engine that guarantees 99.99% application uptime by rerouting around downed AI providers.

Internal Architecture: Implemented in 'framework/circuitbreaker/runtime.go'. Employs a 3-state finite state machine (CLOSED, OPEN, HALF-OPEN). Tracks consecutive 5xx errors, timeouts, and latency spikes across a sliding sample window. When error thresholds are crossed, the circuit TRIPS (OPEN) and diverts traffic to healthy fallbacks.

- Failure Threshold: Configurable error percentage (e.g. 50% errors over 10 requests).
- Latency Threshold: Trips if p95 response time exceeds configured ceiling (e.g. 8000ms).
- Cool-off Duration: Sleep window (e.g. 30s) before transitioning to HALF-OPEN to test provider recovery.
- Fallback Chains: Automatic cascade through secondary and tertiary backup models.
- Header Emission: Returns `x-unifai-fallback-index: 1` to notify callers that a fallback route was executed.

Interconnections: Guards all outbound requests dispatched by Routing Rules and Model Providers.

Production Use Case: When an OpenAI outage causes 504 Gateway Timeouts, UnifAI automatically shifts chatbot traffic to Claude-3.5-Sonnet in <100ms with zero dropped user sessions.

Pricing Overrides (/workspace/custom-pricing/overrides)

Overview: Granular custom token pricing table defining exact \$/1M token costs for enterprise negotiated discount rates and custom fine-tuned models.

Internal Architecture: Maintained in 'framework/configstore'. Evaluated during post-request processing. Matches the executed provider and model against custom price rules, calculating prompt cost, completion cost, and cached token read cost down to micro-cents.

- Input / Output Token Rates: Configurable cost per 1M prompt and completion tokens.
- Cached Token Discount: Custom pricing for prompt cache read hits.
- Scoped Overrides: Define custom rates for specific enterprise accounts or private clusters.

Interconnections: Calculates exact dollar spend recorded in LLM Logs, Budgets & Limits, and Dashboard.

Production Use Case: Enterprises with custom 30% volume discounts from Microsoft Azure input their exact discounted token rates for precise financial accounting.

Model Settings (/workspace/custom-pricing)

Overview: Global routing parameters, request timeout defaults, retry policies, and fallback model definitions.

Internal Architecture: Defines platform-wide default behaviors applied when specific provider or routing rules are omitted. Manages HTTP transport timeouts, max retries, exponential backoff with jitter, and default fallback models.

- Global Timeouts: Sets default request and connect timeouts (e.g. 60s).
- Retry Policy: Configures maximum retry attempts and exponential backoff jitter.
- Default Fallback: Universal emergency fallback model when all routing chains fail.

Interconnections: Underpins the execution engine across all Model Providers.

Production Use Case: Standardizes a 30-second timeout and 2-retry policy across all AI microservices in the organization.

Pillar 3: MCP Gateway (மூலம் & ஏஜென்ட்கள் - Model Context Protocol)

The Model Context Protocol Gateway standardizes agentic tool integration, sandboxed code execution, and enterprise OAuth credential delegation.

MCP Catalog (/workspace/mcp-registry)

Overview: Master registry of all registered MCP servers, available tool definitions, schemas, and transport configurations.

Internal Architecture: Located in 'core/mcp'. Implements the Model Context Protocol JSON-RPC 2.0 specification over stdio, SSE, and in-process transports. Maintains active connections, fetches available tool schemas, resources, and prompt templates, and dynamically formats tool specifications into model-native tool calling schemas.

- Server Registry: Catalog of active and registered MCP servers.
- Schema Inspector: View function signatures, descriptions, and required JSON parameters.
- Transport Support: Native support for stdio subprocesses and HTTP Server-Sent Events (SSE).
- Health Monitor: Continuously pings MCP servers and flags unresponsive processes.

Interconnections: Supplies tool definitions to Tool Groups and the LLM execution pipeline; records execution in MCP Logs.

Production Use Case: Agent developers inspect what database query tools and web scraping functions are currently registered and online.

MCP Library (/workspace/mcp-registry/library)

Overview: One-click curated installer for popular, production-ready open-source MCP servers.

Internal Architecture: Pre-configured repository of enterprise MCP servers. Allows administrators to install and spin up containerized or subprocess MCP servers with pre-tested configurations in a single click.

- Curated Servers: GitHub, PostgreSQL, Slack, Google Drive, Jira, Local Filesystem, Memory, Brave Search.
- Configuration Templates: Pre-filled environment variables and connection string templates.
- 1-Click Deploy: Automatically provisions and connects servers to the MCP Catalog.

Interconnections: Directly registers newly installed tools into the MCP Catalog and Tool Groups.

Production Use Case: A team instantly adds a PostgreSQL database query tool to their AI Agent in 30 seconds without writing custom integration code.

Tool Groups (/workspace/mcp-tool-groups)

Overview: Logical grouping and security boundary mechanism bundling related tools together for role-based assignment to Virtual Keys.

Internal Architecture: Maintained in 'framework/mcptoolgroups'. Groups individual tools into logical operational bundles (e.g. 'DevOps-Tools', 'Finance-Tools', 'Support-Tools'). Attached to Virtual Keys via Access Profiles to enforce strict least-privilege tool access.

- Tool Bundles: Logical collections of tools from one or more MCP servers.
- Granular Whitelisting: Selectively include or exclude individual functions within an MCP server.
- Access Profile Mapping: Bind tool groups directly to specific Virtual Keys.

Interconnections: Bound to Virtual Keys via Access Profiles; controls which tools are injected into LLM requests.

Production Use Case: Ensures a Customer Support bot only has access to Zendesk tools, completely blocking it from executing Database Drop Table commands.

Auth Sessions (/workspace/mcp-sessions)

Overview: Stateful credential manager tracking per-user authenticated sessions for tools requiring individual user permissions.

Internal Architecture: Located in 'core/mcp/credstore'. Associates stateful credentials (e.g. personal access tokens, session tokens) with individual human users via 'x-uf-mcp-session-id'. When an agent calls a tool, UnifAI injects that specific user's credentials rather than a shared corporate master key.

- Session Keying: Keyed to an end-user SSO identity, virtual key, or client session ID.
- Credential Vault: Encrypted at-rest storage for user session tokens.
- Session Lifecycle: Automatic expiration, idle timeouts, and manual revocation.

Interconnections: Works alongside OAuth Grants to supply credentials during MCP execution; logged in MCP Logs.

Production Use Case: When an executive asks an AI agent to 'Summarize my unread emails', the agent accesses Gmail using that executive's specific session token.

OAuth Grants (/workspace/oauth-grants)

Overview: Downstream OAuth 2.0 authorization server managing consent screens, token exchange, and refresh token cycles for 3rd-party tools.

Internal Architecture: Implemented in 'framework/oauth2'. Acts as a full OAuth 2.0 authorization server supporting PKCE. Handles authorization code redirects, user consent interfaces, token storage, and automated background refresh token cycles for third-party providers (Google, Microsoft, GitHub, Slack).

- OAuth 2.0 PKCE Flow: Secure authorization flow for browser and desktop clients.
- Consent Management: Displays fine-grained scope approval screens to end users.
- Token Refresh Engine: Automatically refreshes expired OAuth access tokens in the background.

Interconnections: Supplies live access tokens to Auth Sessions for MCP tool execution.

Production Use Case: End-users securely authorize an AI agent to read their Google Calendar without ever exposing their Google password.

MCP Settings (/workspace/mcp-settings)

Overview: Global execution parameters for MCP servers, including execution timeouts, concurrency limits, and restart policies.

Internal Architecture: Configures runtime constraints for MCP tool execution processes. Enforces hard execution timeouts, maximum concurrent tool processes, memory limits, and auto-restart policies.

- Execution Timeout: Maximum duration a tool is permitted to run (e.g. 30s) before cancellation.
- Concurrency Limits: Caps simultaneous tool executions to prevent server CPU exhaustion.
- Process Restart Policy: Auto-restart crashed stdio server subprocesses.

Interconnections: Governs the execution runtime of all servers in the MCP Catalog.

Production Use Case: Terminates rogue or hanging tool execution scripts after 30 seconds to prevent resource exhaustion.

Plugins (/workspace/plugins)

Overview: Modular extensibility framework supporting custom Go plugins and sandboxed Starlark scripts in the request pipeline.

Internal Architecture: Located in 'core/pluginpipeline.go' and 'core/mcp/codemode/starlark'. Supports both compiled Go plugins implementing BasePlugin and dynamically interpreted Starlark scripts. Provides lifecycle hooks: PreLLMHook, PostLLMHook, PreMCPHook, and PostMCPHook.

- Lifecycle Hooks: Intercept and modify requests before and after LLM and MCP execution.
- Starlark Script Sandbox: Secure, non-Turing complete runtime for dynamic in-memory transformations.
- Custom Encryption: Apply proprietary data encryption algorithms to prompts.

Interconnections: Integrates into the core FastHTTP request-response lifecycle.

Production Use Case: An enterprise injects a proprietary data anonymization algorithm into the PreLLMHook before prompts leave the company network.

Pillar 4: Governance & Enterprise Compliance (அணுகல் கட்டுப்பாடு & பாதுகாப்பு)

The Governance pillar establishes multi-tenant identity, role-based access control, automated SCIM provisioning, and regulatory audit logging.

Virtual Keys (/workspace/governance/virtual-keys)

Overview: Synthetic proxy API keys (sk-uf-*) issued to applications, encapsulating permissions, quotas, and upstream credential masking.

Internal Architecture: Located in 'plugins/governance/main.go'. Virtual Keys are high-entropy cryptographic strings starting with 'sk-uf-'. They hide upstream provider credentials, binding the request to a specific Team, Customer, Budget Limit, Access Profile, and Tool Group. Verified in <0.5ms via local cache.

- Key Obfuscation: Completely hides sensitive upstream OpenAI and Anthropic master keys.
- Configurable Expiration: Set automated key expiry dates (e.g. 90-day rotation).
- IP & CIDR Restrictions: Restrict key usage to specific corporate IP address ranges.
- Instant Revocation: Revoke compromised keys in real-time across the cluster without restarting apps.

Interconnections: Entry point for authentication; enforces Budgets & Limits, Access Profiles, and Audit Logs.

Production Use Case: A microservice uses 'sk-uf-prod-svc-123' to make AI calls; if compromised, it can be revoked instantly without affecting other services.

Users (/workspace/governance/users)

Overview: User account directory for developers, administrators, and stakeholders with workspace access.

Internal Architecture: Manages platform user accounts, authentication credentials (bcrypt hashed), SSO SAML/OIDC identity bindings, and Multi-Factor Authentication (MFA) status.

- User Directory: Searchable list of workspace members and their status.
- MFA Enforcement: Mandatory Two-Factor Authentication via TOTP authenticator apps.
- Session Invalidation: Force-terminate active user login sessions across devices.

Interconnections: Members of Teams; assigned Roles & Permissions (RBAC); actions tracked in Audit Logs.

Production Use Case: Workspace administrators invite developers and enforce MFA for all production environment access.

Teams (/workspace/governance/teams)

Overview: Departmental organizational units managing shared budgets, team-owned virtual keys, and delegated administration.

Internal Architecture: Organizes users into functional departments (e.g. Frontend Engineering, Data Science, Customer Support). Virtual Keys and Budgets are created at the Team level for unified departmental tracking.

- Team Hierarchy: Departmental groupings with dedicated Team Admins.
- Shared Budget Pool: Shared monthly financial spending quota across team members.
- Team Virtual Keys: Keys accessible only by authorized team engineers.

Interconnections: Groups Users; belongs to Business Units; owns Virtual Keys and Budgets.

Production Use Case: The Data Science team is allocated a \$5,000 monthly budget and 5 dedicated GPU model keys.

Business Units (/workspace/governance/business-units)

Overview: Top-level enterprise divisions grouping multiple teams for macroscopic financial chargeback and corporate reporting.

Internal Architecture: Represents top-level corporate subsidiaries or operating divisions (e.g. Retail Banking, Commercial Insurance, Corporate IT). Aggregates token consumption across multiple child teams.

- Division Grouping: Hierarchical aggregation of teams.
- Corporate Chargeback: High-level financial reporting for CFO and Finance teams.

Interconnections: Contains multiple Teams; reported in Dashboard and BigQuery connector exports.

Production Use Case: Corporate Finance views the total quarterly AI expenditure of the entire Retail Division across 15 engineering teams.

Customers (/workspace/governance/customers)

Overview: Multi-tenant client registry for B2B SaaS applications, tracking token consumption and cost per paying customer.

Internal Architecture: Allows B2B SaaS platforms to track external tenant consumption. When requests include 'x-uf-customer-id', UnifAI isolates and attributes token counts and dollar spend to that specific customer entity.

- Customer Registry: Directory of client organizations using the SaaS product.
- Per-Customer Quotas: Enforce rate limits and cost caps on specific customer accounts.
- Billing Reports: Export itemized monthly AI consumption bills per customer.

Interconnections: Scoped via the 'x-uf-customer-id' header; reported in Dashboard and LLM Logs.

Production Use Case: A legal SaaS platform calculates that Customer X consumed \$412 in GPT-4o legal doc summaries this month, auto-invoicing them via Stripe.

User Provisioning (SCIM) (/workspace/scim)

Overview: SCIM v2.0 endpoint automating employee onboarding and deprovisioning from enterprise Identity Providers (Okta, Azure AD).

Internal Architecture: Implements RFC 7643 and RFC 7644 SCIM 2.0 endpoints (/scim/v2/Users, /scim/v2/Groups). Receives webhook push updates from enterprise IdPs (Okta, Microsoft Entra ID / Azure AD, PingIdentity). Automatically provisions accounts, assigns team memberships, and immediately deactivates former employees.

- SCIM 2.0 Protocol: Standardized enterprise user lifecycle integration.
- Automated Deprovisioning: Instant revocation of platform access when an employee leaves the company.
- Group Mapping: Automatically maps corporate Okta groups to UnifAI Teams and Roles.

Interconnections: Directly provisions Users and updates Team memberships; actions recorded in Audit Logs.

Production Use Case: When an engineer is offboarded in Okta, their UnifAI account and personal virtual keys are deactivated within 1 second.

Roles & Permissions (RBAC) (/workspace/governance/rbac)

Overview: Fine-grained Role-Based Access Control matrix governing administrative actions across all gateway resources.

Internal Architecture: Implemented in 'framework/rbac'. Enforces a matrix of Resources (VirtualKeys, RoutingRules, Guardrails, Logs, Providers, Settings) and Operations (View, Create, Edit, Delete). Evaluated on every UI route match and control plane API call.

- Predefined Roles: Super Admin, Workspace Admin, Developer, Auditor, Viewer.
- Custom Roles: Create bespoke enterprise roles with granular permission toggles.
- Resource Scoping: Restrict permissions to specific Teams or Business Units.

Interconnections: Governs UI visibility (via useRbac hook) and authorizes Admin API calls.

Production Use Case: Junior developers can view logs and test prompts, but are blocked from creating new Virtual Keys or modifying Routing Rules.

Access Profiles (/workspace/governance/access-profiles)

Overview: Reusable security policy presets defining allowed models, providers, and MCP tool groups bound to Virtual Keys.

Internal Architecture: Located in 'framework/configstore'. Encapsulates model and tool permissions into reusable bundles. When attached to a Virtual Key, UnifAI rejects any request attempting to use an unapproved model or unlisted MCP tool.

- Approved Models Whitelist: Exact list of allowed model IDs.
- Approved Providers Whitelist: Approved vendor endpoints.
- Approved Tool Groups: Permitted MCP tool groups.

Interconnections: Bound to Virtual Keys; evaluated during Stage 1 request validation.

Production Use Case: A 'Staging-Profile' guarantees that staging microservices can only use cheap open-source models (Llama 3.1 8B), preventing accidental GPT-4o spend in QA.

Audit Logs (/workspace/audit-logs)

Overview: Cryptographically timestamped, immutable audit ledger recording all administrative mutations for SOC 2 and ISO 27001 compliance.

Internal Architecture: Every state mutation across the platform (creating a key, editing a route, altering budget limits) triggers an immutable audit event.

Captures Actor ID, action type, target resource, JSON diff of changes (before/after), source IP address, user agent, and timestamp.

- Immutable Storage: Tamper-resistant append-only database table.
- JSON Diff Inspector: Visual before-and-after comparison of configuration changes.
- Compliance Export: One-click export to CSV / JSON for regulatory audits.

Interconnections: Records all mutations across Governance, Models, Guardrails, and Settings; streams to Connectors.

Production Use Case: SOC 2 auditors verify who changed the production routing rule on August 12th and view the exact JSON diff of the modification.

Pillar 5: Guardrails & Content Security (உள்ளடக்கப் பாதுகாப்பு & களங்கம்)

The Guardrails pillar intercepts prompt inputs and model outputs to enforce PII redaction, brand safety, prompt injection defense, and cluster sync.

Rules (/workspace/guardrails/configuration)

Overview: Safety policy definitions executed pre-LLM (input validation) and post-LLM (output validation) to mask PII and block prompt injections.

Internal Architecture: Located in 'plugins/guardrails/main.go'. Pre-compiles Google CEL expressions for lightning-fast execution. Evaluates input prompts before they reach external LLMs and output responses before they return to clients. Supported actions: Block request (400 Bad Request), Redact/Mask matching tokens (e.g. '[REDACTED_SSN]'), or Flag in logs.

- Pre-LLM Rules: Input scanning for prompt injection, jailbreak keywords, and leaked secrets.
- Post-LLM Rules: Output scanning for hallucinated API keys, toxic output, or leaked PII.
- Custom Actions: Block, Redact/Mask, Flag, or Replace.
- Threshold Tuning: Custom sensitivity scores for classification models.

Interconnections: Executes in Stage 2 (Pre-LLM) and Stage 6 (Post-LLM); annotates records in LLM Logs.

Production Use Case: Automatically detects and masks credit card numbers and Aadhaar numbers with '[REDACTED_PII]' before sending prompts to OpenAI.

Providers (/workspace/guardrails/providers)

Overview: Configuration console for specialized safety engines, including Presidio DLP, Llama Guard, AWS Bedrock Guardrails, and regex filters.

Internal Architecture: Adapters integrating dedicated detection engines: 1. Microsoft Presidio DLP (entity extraction for PII), 2. Meta Llama Guard (content moderation classification), 3. AWS Bedrock Guardrails (managed cloud safety), 4. Lakera AI (prompt injection & jailbreak defense), and 5. High-speed compiled regex engines.

- Presidio DLP Integration: Recognizes SSN, Credit Cards, Names, Phone Numbers, Email, Medical IDs.
- Llama Guard Integration: Flags hate speech, violence, self-harm, and sexual content.
- Regex Fast-Path: Zero-latency pattern matching for proprietary internal account numbers.

Interconnections: Supplies detection capabilities to Guardrail Rules and Browser AI.

Production Use Case: Enables Meta Llama Guard to block adversarial jailbreak attempts ('Ignore all previous instructions...').

Cluster Config (/workspace/cluster)

Overview: Multi-node distributed cluster coordination engine synchronizing rules, rate limit counters, and configurations in real-time.

Internal Architecture: Coordinates distributed UnifAI gateway instances deployed in high-availability clusters. Uses pub/sub sync to propagate routing rule updates, newly issued virtual keys, and rate limit counters across all nodes in <50ms without restarting services.

- Node Discovery: Real-time health monitoring of all active cluster gateway instances.
- Config Synchronization: Instant zero-downtime propagation of rule mutations.
- Distributed Rate Limiting: Synchronized Redis counters preventing quota circumvention.

Interconnections: Synchronizes Governance, Routing Rules, Guardrails, and Caching across all cluster nodes.

Production Use Case: A global deployment with 10 gateway instances across US-East and EU-West maintains synchronized rate limits and instant rule updates.

Pillar 6: Adaptive Routing & Assets (ஸ்தார்ட் தரேவ் & அறிவு களஞ்சியம்)

Adaptive Routing leverages reinforcement learning to optimize latency and cost while managing versioned prompt templates and agent skills.

Adaptive Routing Dashboard & Settings (/workspace/adaptive-routing)

Overview: Reinforcement learning traffic router that continuously evaluates provider latency and error rates to steer requests to the fastest, cheapest model.

Internal Architecture: Implements Multi-Armed Bandit (MAB) optimization algorithms (Epsilon-Greedy, Thompson Sampling, Upper Confidence Bound UCB1).

Continuously evaluates real-time latency, error rates, and token costs across equivalent candidate models. Dynamically shifts traffic weights to minimize p95 latency while adhering to cost budgets.

- Algorithm Selection: Toggle between Epsilon-Greedy, Thompson Sampling, and Latency-Weighted routing.
- Latency vs Cost Sliders: Configurable weighting favoring speed vs dollar savings.
- Exploration Ratio: Percentage of traffic reserved for testing slower/newer models.
- Performance Dashboard: Visual charts showing dynamic weight distribution and latency improvements.

Interconnections: Acts as an intelligent layer within Routing Rules; reads performance telemetry from LLM Logs.

Production Use Case: When Azure OpenAI experiences temporary latency spikes (p95 > 4000ms), the Adaptive Router automatically diverts 90% of traffic to AWS Bedrock.

Prompt Repository (/workspace/prompt-repo)

Overview: Centralized, version-controlled enterprise prompt repository supporting variable interpolation, test suites, and model bindings.

Internal Architecture: Enterprise prompt management engine. Stores parameterized prompt templates with Git-style versioning (v1, v2, production tag). Supports Mustache/Jinja variable interpolation ('{{customer_query}}'), parameter presets (temperature, top_p), and test suite benchmarking.

- Version Control: Full commit history, diffs, and instant rollback capabilities.
- Variable Templating: Define required variables with default values and type validation.
- Model Binding: Associate prompts with tested, approved model architectures.
- Playground Testing: Interactive test console to execute templates against real models.

Interconnections: Referenced by applications via prompt IDs; logged in LLM Logs; bound to specific Model Catalog entries.

Production Use Case: Prompt engineers update customer service prompt templates in UnifAI; microservices automatically consume the updated prompt without code redeployments.

Skills Repository (/workspace/skills-repo)

Overview: Standardized catalog of autonomous AI Agent instructions, domain personas, and specialized execution skill packages.

Internal Architecture: Stores modular, reusable agent skills (system instructions, role definitions, operational guardrails). Skills can be dynamically injected into LLM requests or bound to specific MCP Tool Groups to build autonomous agents.

- Skill Packages: Modular instructions (e.g. 'SQL-Expert', 'Python-Debugger', 'Compliance-Auditor').
- Tool Bindings: Associate specialized skills directly with approved MCP tools.
- Version History: Track refinements in agent behavior and system instructions.

Interconnections: Combines with MCP Tool Groups and Prompt Repository to configure autonomous agents.

Production Use Case: Teams share a battle-tested 'Secure Code Reviewer' skill package across 20 different development squads.

Pillar 7: Global Settings & Engine Tuning (கட்டமைப்பு & சயெல்திறன் ட்யூனிங்)

Platform-wide system configuration, wire compatibility translation, semantic vector caching, and network security.

Client Settings (/workspace/config/client-settings)

Overview: Global connection timeouts, keep-alive settings, and header forward allowlists.

Internal Architecture: Configures network-level HTTP client behavior in FastHTTP. Defines header forwarding allowlists and blocklists, controlling which client headers are forwarded to upstream AI providers.

- Header Allowlists: Explicitly permit custom headers (x-uf-eh-*) to reach upstream providers.
- Header Blocklists: Strip sensitive internal tracking headers before dispatch.
- Keep-Alive Tuning: Configure client connection persistence and idle socket timeouts.

Interconnections: Governs the inbound and outbound HTTP Transport layer.

Production Use Case: Allows microservices to forward custom transaction correlation IDs to upstream LLM providers while stripping internal cookies.

Compatibility (/workspace/config/compatibility)

Overview: Automated JSON API translation engine bridging OpenAI, Anthropic, AWS Bedrock, and Google Gemini schemas.

Internal Architecture: Located in 'plugins/compat'. Real-time schema converter. Enables client applications written with the OpenAI SDK ('/v1/chat/completions') to interact seamlessly with Anthropic ('/v1/messages'), AWS Bedrock Converse, or Google Vertex AI without modifying a single line of client code.

- Schema Translation: Converts messages, roles (system/user/assistant), tools, and choices across provider formats.
- Parameter Adaptation: Automatically converts or safely drops unsupported parameters (e.g. frequency_penalty).
- Streaming SSE Normalization: Normalizes provider-specific SSE streaming chunks into standard OpenAI-compatible chunks.

Interconnections: Executes inside the HTTP Transport layer before provider dispatch and after response receipt.

Production Use Case: An application built exclusively for OpenAI can switch seamlessly to Anthropic Claude 3.7 Sonnet by changing only the model name.

Caching (Semantic Cache) (/workspace/config/caching)

Overview: High-speed semantic vector caching engine using Redis or Qdrant to deliver instant, zero-cost responses for similar queries.

Internal Architecture: Implemented in 'plugins/semanticcache'. Combines direct SHA-256 hash lookup with vector embedding search. Generates vector embeddings of user queries using fast embedding models (OpenAI text-embedding-3-small or local BGE). Performs cosine similarity search against Qdrant / Redis vector partitions. If similarity $\geq$ threshold (default 0.90), returns cached answer in <15ms at \$0.00 cost.

- Vector Store Backend: Connect to Redis Vector Engine or Qdrant.
- Embedding Generator: Configurable embedding model source for vectorizing incoming prompts.
- Similarity Threshold: Global cosine similarity threshold (e.g. 0.90).
- Partition Keying: Scopes cache entries via 'x-uf-cache-key'.
- Streaming Cache: Caches streaming SSE chunks for seamless instant replay.

Interconnections: Evaluated in Stage 3 of the request pipeline; records cache hit savings in Dashboard and LLM Logs.

Production Use Case: In high-volume customer support, 25% of queries are repeated questions; Semantic Caching answers them instantly, cutting monthly API spend by thousands of dollars.

Security (/workspace/config/security)

Overview: Enterprise network defense console managing TLS certificates, IP allowlists, CORS policies, and secret encryption.

Internal Architecture: Located in 'core/schemas/vault.go' and HTTP middleware. Enforces TLS 1.3 encryption in transit, encrypts API keys at rest using AES-256-GCM, validates incoming IP addresses against CIDR allowlists, and enforces strict Cross-Origin Resource Sharing (CORS) rules.

- AES-256-GCM Encryption: Master key encryption for all stored provider secrets and virtual keys.
- IP CIDR Whitelisting: Restrict gateway access to designated corporate VPN or VPC IP subnets.
- CORS Policy Manager: Whitelist allowed frontend web origins.
- Direct Key Policy: Enable or disable 'x-uf-direct-key' pass-through globally.

Interconnections: Protects all incoming connections and secures all stored secrets in PostgreSQL.

Production Use Case: Security teams restrict UnifAI Gateway access exclusively to internal corporate VPC subnets and enforce TLS 1.3.

API Keys (/workspace/config/api-keys)

Overview: Administrative API key manager for automation, CI/CD pipelines, and infrastructure-as-code orchestration.

Internal Architecture: Issues and manages Control Plane Admin API Keys ('x-uf-api-key'). Used by Terraform providers, Kubernetes operators, and CI/CD pipelines to programmatically configure UnifAI.

- Admin Key Generation: Generate cryptographically random administrative keys.
- Scope Assignment: Assign read-only or read-write permissions to admin keys.
- Usage Tracking: Audit log of actions executed by each admin key.

Interconnections: Authorizes programmatic control-plane configuration requests; recorded in Audit Logs.

Production Use Case: Terraform scripts use an Admin API Key to provision Virtual Keys and Routing Rules during automated environment spin-up.

Performance Tuning (/workspace/config/performance-tuning)

Overview: Low-level runtime performance tuning console for worker pool concurrency, memory pools, and buffer sizes.

Internal Architecture: Located in 'core/unifai.go'. Controls internal Go runtime parameters: FastHTTP request channel pool size (channelMessagePool), response stream pool (responseStreamPool), worker goroutine counts per provider queue, and memory buffer allocations.

- Worker Pool Size: Maximum concurrent worker goroutines per upstream provider queue.
- Zero-Allocation Pool Sizing: Pre-allocated buffer capacity in 'sync.Pool' to eliminate GC pauses.
- Queue Overflow Policy: Block-and-wait vs immediate rejection ('dropExcessRequests') under heavy traffic.

Interconnections: Optimizes the execution performance of the entire core gateway engine.

Production Use Case: Under peak traffic of 50,000 requests/second, performance engineers tune worker pools to achieve zero GC pause times.

Feature Flags (/workspace/config/feature-flags)

Overview: Dynamic runtime feature toggles allowing administrators to enable, test, or disable capabilities without redeploying the gateway.

Internal Architecture: Maintained in 'framework/featureflags'. Real-time in-memory configuration flags synchronized across the cluster. Allows instant activation or deactivation of beta features (e.g. experimental routers, new provider adapters) with zero downtime.

- Dynamic Toggles: Enable or disable platform features in real-time.
- Gradual Rollout: Enable beta capabilities for specific teams or virtual keys.
- Instant Kill-Switch: Immediately disable problematic features without redeploying code.

Interconnections: Controls code execution paths across all gateway modules.

Production Use Case: A platform team safely tests an experimental new reasoning router with internal engineers before enabling it organization-wide.

5. Technology Stack & Programming Languages Deep Dive

Layer	Technology / Language	Architectural Role & Benefit
Core Gateway Engine	Go (Golang 1.23+)	Ultra-low proxy latency (<1ms), lightweight goroutine concurrency, memory safety, and native zero-allocation memory pools (sync.Pool).
HTTP & Networking	FastHTTP & Sonic JSON	FastHTTP avoids per-request heap allocations of standard net/http. ByteDance's Sonic JIT compiler parses JSON at near-native assembly speeds.
Rule Evaluation Engine	Google CEL (Common Expression Language)	Safe, non-Turing complete expression evaluation. Compiles routing rules and guardrail policies into bytecode, executing in microseconds.
Agent Scripting Sandbox	Starlark (Google Bazel Language)	Deterministic, thread-safe, sandboxed Python-like language for executing dynamic custom plugins without security risk.
Frontend Web Application	TypeScript 5.x, React 18, Vite	Strict compile-time type safety across all 38 UI views, instant HMR development, and optimized single-page app bundle delivery.
Routing & UI Components	TanStack Router, Tailwind CSS, Radix UI	TanStack Router provides preload-on-hover routing for instantaneous navigation. Radix UI (Shadcn) delivers accessible, sleek dark-mode components.
State Management	Redux Toolkit (RTK Query), WebSockets	Manages client-side caching, optimistic UI updates, and real-time live metrics streaming via WebSockets.
Desktop / Browser DLP Agent	Python 3.11+, mitmproxy / asyncio	Mature networking and proxy ecosystem for local HTTPS interception, regex matching, and integration with local Ollama models.
Relational Database	PostgreSQL (pgx v5 driver), SQLite, GORM	ACID transactional storage for Governance, Virtual Keys, Audit Logs, and Budgets. pgx provides high-performance connection pooling.
Caching & Vector Store	Redis (go-redis), Qdrant, pgvector	Sub-millisecond sliding-window rate limiting in Redis, and high-throughput vector similarity search for Semantic Caching in Qdrant.
Telemetry & Observability	OpenTelemetry (OTel), Prometheus, Datadog, Kafka	W3C standard distributed tracing, Prometheus metric scrapers, and event-driven enterprise streaming to Kafka and Datadog.
DevOps & Build System	Docker, Docker Compose, Nix / Flake, Makefile	Hermetic, reproducible development and build environments across multi-platform container architectures.

6. Enterprise Production Scenarios & Playbooks

Scenario 1 - Multi-Tenant B2B SaaS Cost Attribution: SaaS backend attaches 'x-uf-customer-id: cust_42' on every call. UnifAI attributes token spend per customer, exported to BigQuery for automated monthly invoicing.

Scenario 2 - Zero-Downtime High Availability Failover: When OpenAI returns 504 errors, Circuit Breaker detects >50% failure rate and automatically shifts traffic to Claude 3.5 Sonnet in <100ms. Zero user sessions drop.

Scenario 3 - Slashing Costs via Complexity Router & Semantic Cache: Semantic Caching serves 25% of queries from Redis at \$0.00 cost; Complexity Router directs simple questions to micro-models, dropping monthly AI bill from \$45,000 to \$9,800 (78% savings).

Scenario 4 - Enterprise Data Loss Prevention (DLP) using Browser AI: Browser AI local agent monitors web AI portals on employee laptops, intercepting clipboard pastes containing credit cards or passwords, preventing corporate IP leaks.